```
    end

    sig { params(word: String).returns(T.nilable(T::Array[String])) }
    def lookup(word)
      signature = signature_of(word)
      @signatures[signature]
    end

    sig { params(word: String).returns(String) }
    def signature_of(word)
      word.unpack("c*").sort.pack("c*")
    end
  end
end
```

There's a lot to look at here, but it's worth emphasizing that the basic idea is the same as with RBS—we're trying to explicitly denote the types of method arguments, return values, and class attributes.

The file itself starts with two pieces of boilerplate: the magic comment `# typed: true`, which tells Sorbet to pay attention to this file, and then the `Finder` class has `extend T::Sig`, which puts Sorbet's type signature methods in the class.

Each individual method gets its type signature specified by prefacing the method with a call to `sig`. The `sig` method is provided by Sorbet, and it takes a block. Inside that block, you can optionally specify the parameters to the method, and you must specify the return value of the method.

The parameters are specified with the `params` method. The `params` method takes keyword arguments, the keys of which are the names of each parameter of the actual method, and the values of which are the type of the method.

For example, the `from_file` method has the `sig` call `sig { params(file_name: String).returns(Aaagmnr::Finder) }`. The `params(file_name: String)` part says that there is a parameter to the method named `file_name` and its expected type is `String`. For Sorbet's purpose, the `params` method has the same structure—parameter name and type—no matter whether the parameter in the actual method is positional or keyword, and no matter whether the argument is required or optional (Sorbet can infer whether the argument is optional from the Ruby code).

One gotcha is that splat and double-splat types are annotated with the type of an individual element of the resulting data structure, not the data structure as a whole:

```
sig { params(args: String, kwargs: String) }
def i_have_splats(*args, **kwargs)
end
```

In this snippet `args` is of type `String`, not an array of strings, and `kwargs` is also of type `String` and not an array with Symbol keys and String values. Sorbet can infer the data structure from the code.

The return value of the method is handled by chaining a call to `returns`. The argument to `returns` is the type returned by the method. If the method's return value isn't used, call `void` instead of `returns`—in the example, the `initialize` method uses `void`.

Sorbet can infer the type of local variables from the initial assignment to the valuable. For constants and for cases where the initial assignment is not enough information, Sorbet would